

Nexus: Investigating Byte-Level Structured Embeddings for Parameter-Efficient Language Models

Rishit Laddha

ATLAS SkillTech University, Mumbai

2309575

Introduction

Every transformer language model begins the same way: text gets split into tokens, and each token is looked up in a table that converts it into a list of numbers the network can actually operate on. This table is learned initialized randomly and adjusted throughout training along with everything else in the model.

At small scale, this table's cost is unremarkable. At the scale modern language models are trained at, it stops being unremarkable a model with a large vocabulary and a wide hidden layer can end up spending a substantial share of its total trainable parameters on this one lookup step, before the network has done anything resembling reasoning. Standard training procedures compound this further, since each trainable parameter carries additional optimizer-state memory alongside it.

The justification for paying this cost is an assumption: that training discovers something in this table worth the expense that the table comes to encode real relationships between words, and the model leans on that structure downstream. This proposal starts by treating that justification as something to check, not something to take for granted, and builds outward from there toward an alternative: a fixed, non-learned rule for constructing a token's starting representation, built directly from the token's raw bytes, paired with one small trainable layer instead of an entire learned table.

The investigation is structured as five claims, in the order they need to be asked, each depending on the one before it.

Claim 1 : trained embedding tables may not encode what's assumed. Before proposing any replacement, the first step is to check what a trained table actually organizes tokens by. If it turns out to already capture rich relationships between words, that's a real cost against replacing it. If it turns out to be doing something narrower, that's what opens the door to the rest of this proposal.

Claim 2 : a fixed, byte-position rule produces its own kind of structure. Rather than leave the replacement as a vague idea, its behavior needs to be pinned down precisely: what does it cluster together, and why. This is where the proposal shifts from a question about existing models to a concrete definition and correspondingly, where it starts needing to lean on math rather than just inspection, since a fixed rule's properties can be worked out on paper rather than discovered empirically.

Claim 3 : the fixed rule should not hurt, and may help, actual model performance. This is the central bet, and the one that would decide whether the previous two claims matter at all. If a model built on the fixed rule performs the same or worse than one built on a learned table, the idea isn't worth its added complexity, however clean the math behind it looks.

Claim 4 : if Claim 3 holds, the fixed rule's mathematical properties should show up as observable model behavior, not just as vector geometry. Specifically, the closed-form prediction about how much a small edit (like a typo) should disturb the encoding derived directly from the definition, not from data gives a concrete claim to test against a trained model's real outputs.

Claim 5 : if Claim 3 holds, the most obvious explanation for why needs to be checked directly rather than assumed. The natural story is that the fixed rule helps the model build better word-relationship understanding early on. That story is testable, and worth testing honestly rather than left implied.

Here you go both sections, written as your own proposal, formulas kept (they're citable), literature cited properly, and deliberately no results-numbers anywhere, since nothing's been run yet.

Background and Related Work

The cost this proposal is responding to is straightforward arithmetic: any transformer's input embedding table scales as vocabulary size times hidden width, and at the scale modern models run at, that single lookup table can account for a substantial fraction of total trainable parameters before any actual reasoning computation happens. Under standard Adam-style training, each of those parameters also carries additional optimizer-state memory on top of itself, so the true cost during training exceeds the raw parameter count.

Several existing directions attempt to reduce this cost, each leaving something on the table that this proposal is built around.

Factorized embeddings. Lan et al. (2020), in ALBERT, factor the embedding matrix into two smaller matrices multiplied together, reducing the parameter count from vocabulary-size-times-width down to two much smaller learned factors. This is a genuine reduction, but both factors remain trained the underlying assumption that the representation must be learned from data is untouched.

Weight tying. Press and Wolf (2017) proposed sharing the input embedding and output projection matrices, saving an entire matrix's worth of parameters by having one do double duty. This became a standard choice in many small and mid-sized models. More recently, Lopardo et al. (2026) argue this sharing has a hidden cost: because the output side receives a stronger gradient signal during training, a tied matrix tends to drift toward serving the output-prediction task rather than representing the input well which is part of why several frontier-scale models have moved to untied embeddings independently of anything proposed here.

Tokenizer-free and byte-level approaches. A separate line of work removes the tokenizer entirely and processes raw text at the byte or character level directly CANINE (Clark et al., 2022) and ByT5 (Xue et al., 2022) both operate on raw Unicode input rather than subword tokens, and related work (Charformer, MEGABYTE, the Byte Latent Transformer) builds increasingly elaborate architectures around byte-level or dynamically-segmented input. This buys real robustness to spelling and formatting noise, since there's no fixed vocabulary boundary to fall outside of, but it comes at a real sequence-length cost text that a subword tokenizer compresses efficiently expands substantially once processed byte-by-byte. An older conceptual ancestor, Kim et al. (2016), builds token representations from a character-level convolution rather than a lookup table, though on an RNN-based architecture rather than a transformer.

A closer relative: morphology-aware byte encoding. MYTE (Limisiewicz et al., 2024) constructs a byte-level encoding specifically designed around morphological and

multilingual-fairness considerations motivated by the observation that byte-per-token cost varies wildly and unevenly across languages and scripts. This sits conceptually near what this proposal is testing, though the design goals diverge: MYTE reshapes the encoding itself to equalize cross-lingual cost, while the approach here keeps a fixed, uniform byte-position encoding regardless of script.

What none of these fully address. None of the parameter-reduction approaches above remove the assumption that the input representation has to be learned. None of the tokenizer-free approaches preserve a standard tokenizer's sequence-length efficiency. This proposal keeps the tokenizer and sequence length exactly as they are, and only replaces the lookup step itself with a fixed, non-learned rule a narrower, more surgical change than either existing direction, chosen specifically so that a later controlled comparison can isolate the embedding scheme as the one variable under test, without also disturbing tokenization or sequence handling.

METHODOLOGY

The proposed encoding

Instead of learning a token's starting representation from data, the proposal is to compute it deterministically, directly from the token's raw UTF-8 bytes and their positions within the token. For a token with byte sequence $\mathbf{b} = (b_1, \dots, b_L)$:

$$\kappa(\mathbf{b}) = \frac{1}{\sqrt{L}} \cdot \sum_{p=1}^L c_{b_p} \otimes p_p$$

where c_{b_p} is a one-hot vector marking byte value v (256 possible values, one slot per byte), p_p is a one-hot vector marking position p within the token (up to some fixed maximum length d_p), and $\otimes$ combines the two into a single vector marking that specific (value, position) pair. The result is a fixed-length vector, $D = 256 \times d_p$ dimensions wide, with one nonzero entry per byte in the token.

Nothing in this definition is learned or dependent on training data it is fully determined by the token's own bytes. The only trainable component sits downstream of it: a single linear projection resizing this fixed vector to whatever hidden width the transformer body expects.

Two properties that need to be established before anything is trained

Before this encoding is trusted enough to build a training comparison on top of, two of its properties need to be worked out from the definition itself, since if they don't hold cleanly the rest of the proposal doesn't stand on solid ground.

Consistent scale across tokens. Because no two positions within a token can ever mark the same output slot the slot is jointly determined by value and position, and a token never has two different bytes occupying one position every mark in the sum points in a distinct direction from every other mark. This means the vector's total length, before any further adjustment, comes out to $\sqrt{L}$, and after dividing by $\sqrt{L}$, exactly 1, for every token regardless

of length. This removes any dependence on token length before the vector ever reaches the trainable projection.

A closed-form prediction for small edits. The same non-collision property gives a direct formula for how similar two same-length tokens' vectors will be, given how many byte positions they differ at:

$$\cos(\kappa(b), \kappa(b')) = \frac{(L - k)}{L}$$

for two length- L tokens differing at k byte positions. This gives a concrete, checkable prediction how much a single-character edit should disturb the encoding derivable before any model is trained, and something a later experiment can test directly against a trained model's actual behavior.

After the fixed encoding is computed, it is rescaled to have zero mean and unit variance across its coordinates (per-token normalization), which standardizes the input the trainable projection sees regardless of how many of the vector's coordinates are nonzero for a given token this matters because shorter tokens naturally populate fewer coordinates than longer ones before this step.

Parameter accounting, framed honestly in advance

Because the fixed encoding's dimension generally won't match the transformer's hidden width, the input and output sides of the model can no longer share a single matrix the way a conventional tied embedding table can an untied output matrix becomes mandatory. The parameter cost of this proposal, once an actual vocabulary size and model width are fixed for the experiment, needs to be reported two ways rather than one: the narrow comparison (fixed-encoding projection alone versus the learned table alone) and the full comparison (including the now-mandatory separate output matrix the fixed-encoding side has to carry). Reporting only the narrower framing risks overstating the parameter savings, particularly at smaller scales where the mandatory extra output matrix can outweigh the savings on the input side entirely this is a real possibility this proposal intends to check honestly once the actual model dimensions are fixed, not something assumed away in advance.